

Go-Kafka: Order Ecosystem

Implementation of Transaction Outbox Pattern with PostgreSQL & sqlx

Golang

Apache Kafka

PostgreSQL

sqlx

1. Konsep: Transaction Outbox Pattern

Pola ini digunakan untuk menjamin data tetap konsisten antara database (Postgres) dan broker pesan (Kafka). Dengan membungkus penyimpanan data *order* dan data *outbox* dalam satu **Database Transaction**, kita memastikan tidak ada pesan yang hilang jika Kafka sedang down.

2. Skema Database

```
-- Tabel Utama
CREATE TABLE orders (
    id UUID PRIMARY KEY,
    item_name VARCHAR(255),
    amount DECIMAL,
    status VARCHAR(50), -- PENDING, PAID, FAILED
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabel Outbox (Antrean Pesan Lokal)
CREATE TABLE outbox (
    id BIGSERIAL PRIMARY KEY,
    topic VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3. Implementasi Kode Go (sqlx)

A. Transaksi Atomik (Order + Outbox)

```
func CreateOrder(db *sqlx.DB, order Order) error {
    tx, err := db.Beginx()
    if err != nil {
        return err
    }

    // Insert ke tabel Orders
    _, err = tx.Exec("INSERT INTO orders (id, item_name, ...) VALUES ($1, $2, ...)",
        order.ID, order.Item, ...)

    // Insert ke tabel Outbox (Payload JSON)
    payload, _ := json.Marshal(order)
    _, err = tx.Exec("INSERT INTO outbox (topic, payload) VALUES ($1, $2)",
        "order-events", payload)

    if err != nil {
        tx.Rollback()
        return err
    }
    return tx.Commit()
}
```

B. Outbox Relay (Background Worker)

```
func StartOutboxRelay(db *sqlx.DB, writer *kafka.Writer) {
    for {
        var messages []Outbox
        db.Select(&messages, "SELECT id, payload FROM outbox LIMIT 10")

        for _, msg := range messages {
            err := writer.WriteMessages(context.Background(), kafka.Message{
                Value: msg.Payload,
            })
            if err == nil {
                db.Exec("DELETE FROM outbox WHERE id = $1", msg.ID)
            }
        }
        time.Sleep(2 * time.Second)
    }
}
```

4. Alur Event-Driven Lengkap

Tahap	Service	Aksi
1	Order Service	Simpan DB & Outbox (Atomik)
2	Relay Worker	Kirim dari Outbox ke Kafka
3	Payment Service	Proses Bayar & Update Status
4	Notification	Kirim Email Konfirmasi

Catatan: Pastikan Kafka UI berjalan untuk memantau trafik pesan di port 8080.